Advanced Data Mining: Homework set 8

2025/2026

Problem 1 (5p)

Encoder-Decoder vs. Decoder-Only: The original Transformer (Vaswani et al. 2017) uses an encoder-decoder structure. However, for forecasting, sometimes a decoder-only (auto-regressive) transformer is used. Briefly discuss the difference and why a decoder-only Transformer can be sufficient for time series forecasting.

A. (0.5p) Data Preparation: Load the weather time series dataset (see previous exercise). This is likely a multivariate time series (e.g., temperature, pressure, humidity, etc.).

B. (0.5p) Transformer Model Implementation: Construct a transformer-based model for time series forecasting. You have flexibility in design: One approach is to use an encoder-decoder structure: the encoder processes the past 30 days of multivariate data, and the decoder generates the next 7 days of the target (with appropriate masking so that at training time the decoder can attend to previous target values). Alternatively, implement a decoder-only transformer that takes the 30-day history as context and autoregressively predicts each step up to 7 days (feeding back predictions, using masked self-attention to prevent seeing future data). Use library components where possible (e.g., torch.nn.Transformer or Keras MultiHeadAttention layers) rather than coding attention from scratch. Include positional encoding in your model. For instance, in PyTorch you can use nn.Transformer which includes embedding + positional encoding, or manually add sinusoidal positional features to inputs.

C. (0.5p) Training: Train the Transformer model on the training set. Because transformers have many parameters, use techniques to aid learning: e.g., a relatively large batch size (if data allows), learning rate scheduling, and possibly more epochs. Monitor validation loss for hyperparameter tuning. Training might be slower than RNNs, so be mindful of training time.

D. (0.5p) LSTM Benchmark Model: Using your knowledge from previous list 1, set up a sequence-to-sequence LSTM model for the same task.

E. (1p) Evaluation – Forecast Accuracy: Evaluate both models (Transformer and LSTM) on the test set by producing 7-day forecasts for each window and comparing to actual values. Use metrics suitable for multi-step forecasts, e.g., Mean Absolute Error and RMSE for each horizon step and/or overall (you can compute an average MAE over all forecasted points, and also check MAE at day +1, +3, +7 separately to see if error grows). If the dataset is multivariate and you used additional features, make sure to also feed the necessary exogenous features to the models at prediction time (if required by your model design).

F. (0.5p) Evaluation – Computational Performance: Record the training time or epochs needed for each model to converge. Also, note the number of parameters in each model. This information will feed into a comparison of training efficiency and model complexity.

G. (0.5p) Hyperparameter Tuning: Experiment with at least one or two hyperparameters on the Transformer. For instance, try changing the number of attention heads (e.g., 4 vs 8 heads) or the depth (number of Transformer layers). Observe the effect on validation performance and training time. You might also try different sequence lengths for input or different forecast horizon lengths to see how the transformer handles them versus the LSTM.

H. (1p) Attention Visualization: For one example in the test set, visualize the Transformer's attention weights. For instance, pick a specific forecast date and plot the attention scores of the Transformer decoder for that prediction with respect to the input timeline (and/or previous outputs). This could be done by extracting the attention matrix from the model (in PyTorch's nn.Transformer, you can register hooks or use the attn_output_weights if using MultiheadAttention modules). The goal is to interpret which past time steps the model found relevant for predicting a particular future step. Include a brief analysis of this: do the attention weights align with intuitive patterns (e.g., focusing on daily patterns or recent trends)?

Problem 2 (5p)

A. (0.5p) Select Dataset: Choose a real-world healthcare time series dataset suitable for a classification or anomaly detection task. Examples: an ECG dataset where each time series is labeled as normal or arrhythmia (e.g., the ECG200 dataset from the UCR archive, which contains 200 heartbeat time series for

two classes), or a clinical dataset where sequences of patient vital signs are labeled by outcome. Ensure the data is preprocessed (e.g., each series has the same length or you truncate/zero-pad as needed, and consider normalization).

B. (0.5p) Install and Configure Models: Use official or existing implementations for TS2Vec and T-Rep (see the notebooks presented during the lecture). Make sure you understand the input format each expects (TS2Vec may require creating augmented views; T-Rep's API might handle that internally). If needed, use a smaller subset of data to tune any hyperparameters due to time constraints.

C. (0.5p) Unsupervised Training: Train the TS2Vec model on the training portion of your dataset to learn representations. This involves feeding the raw time series (without labels) into TS2Vec's training routine. Similarly, train T-Rep on the same training data (T-Rep will learn time-step level embeddings and an encoder). Both models will output some form of encoded representation for each time series or each time step. For consistency, decide on the representation you will extract for the downstream task. For example, you might use instance-level embeddings (one vector per entire time series) by aggregating TS2Vec's timestamp embeddings (e.g., average or using the final timestamp) and using T-Rep's output for the final timestamp (or an average as well).

D. (0.5p) Obtain Representations: After training, use the learned models to encode all training and test samples into representation vectors. For TS2Vec, this might mean passing each time series (or sub-sequence) through the trained network to get a sequence of embeddings, then pooling or selecting the relevant part. For T-Rep, you can use trep.encode(data) as in the provided example to get an array of representations. Ensure the dimension of these representations is noted (e.g., TS2Vec default might be 128 or 256 per timestamp, T-Rep default output_dims=128 unless changed).

E. (0.5p) Downstream Classification Task: Using the labeled data, train a simple classifier on top of the learned representations. For example, train a Logistic Regression or Support Vector Machine (SVM) using the instance-level representation of each time series as features to predict its class. (In the ECG example, each heartbeat time series is represented by a learned vector, and you predict "normal" vs "abnormal"). Do this for both TS2Vec-derived embeddings and T-Rep embeddings separately. Evaluate the classification accuracy (or F1-score, etc., depending on the problem) on the test set for each case. If the dataset has a class imbalance, ensure you use appropriate metrics or techniques.

F. (1p) Baseline Comparison: For context, also evaluate a baseline approach. For instance, use raw features or a simpler representation: you could take the raw time series (normalized) and flatten it or use basic statistics (mean, std, etc.) as features for the same classifier. Alternatively, train a fully supervised deep learning model (like a small CNN or RNN) directly on the labeled data for comparison. The goal is to see how the self-supervised representations stack up against a naive representation or a fully supervised approach with the same amount of labeled data.

G. (1.5p) Analysis of Representations: Analyze the quality of the learned representations. Perform the following:

- Visualization: Use a dimensionality reduction technique (e.g., t-SNE or PCA) to project the learned representations of the test set into 2D. Create a scatter plot where each point is a time series in the embedding space, colored by its true label. Do this for TS2Vec and T-Rep embeddings. Discuss any observable clustering – e.g., "We see two distinct clusters for the two classes with T-Rep, indicating good class separation, whereas TS2Vec embeddings overlap more" or vice versa.
- Nearest Neighbors: Pick a few example time series and find their nearest neighbors in the representation space (using Euclidean distance or cosine similarity). Check if those neighbors share the same class or similar characteristics. This can qualitatively show whether the embedding is grouping similar time series together meaningfully.
- Robustness test: If time permits, test robustness to missing data: e.g., remove or mask out a portion of the time series and see how the representations change or if the models can still encode them well (this relates to T-Rep's claim of resilience to missing data.

Dataset Healthcare Time Series (e.g., ECG Classification): ECG200 from UCR Archive – This dataset contains 200 electrocardiogram sequences each of length 96, labeled as either normal or arrhythmia. We will use 100 series for training and 100 for testing (balanced classes). The goal is to classify heartbeats using learned representations.

Alternate options: Another possible dataset is the PhysioNet MIT-BIH Arrhythmia ECG dataset (longer heartbeat signals with multiple classes), or a wearable sensor time series dataset for activity recognition (where each

series is an acceleration signal labeled by activity). Ensure whichever dataset you choose, it has a clear train/test split and labels for a supervised evaluation. (The ECG200 is convenient and small; if a larger challenge is desired, choose a more complex dataset but be mindful of training time for representation learning models.)